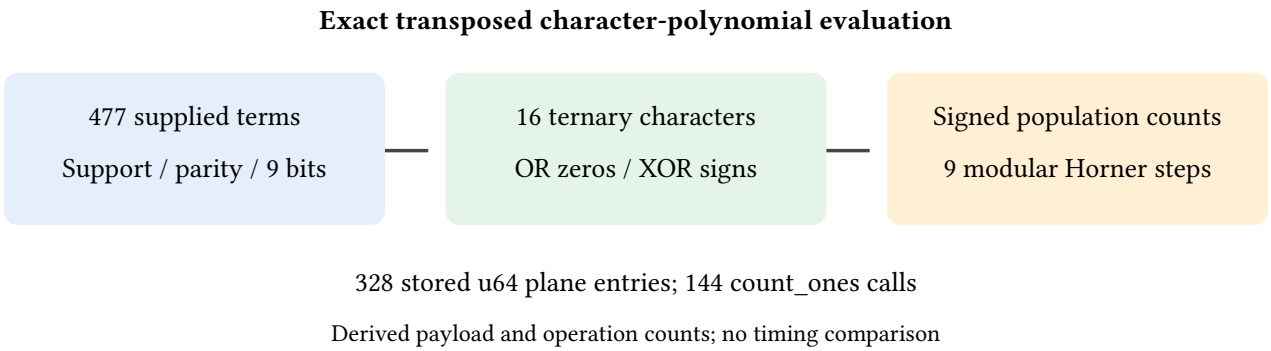
Exact transposed character decoder

2026-11-03 · T-1580

Compile the supplied polynomial across terms into support, parity and coefficient bitplanes. For each coefficient bit, signed popcounts give the sum of surviving positive terms minus negative terms. Descending modular Horner evaluation reconstructs the exact polynomial. Repeated offsets preserve zero sensitivity separately from sign parity.



Construction

For active coefficient bits A and negative-term mask M, the signed digit is popcount(A)-2popcount(A AND M), summed across words. The existing477-term packet uses16 character inputs,9 coefficient bits and8 words per plane:328 u64 payload entries, or2624 bytes. Evaluation uses144 population-count calls and9 modular Horner steps. These are derived counts. Query scratch includes1008 bytes of signed digit storage; no decoder-owned heap allocation occurs. Setup transposition, container metadata and the source's costs are separate.

Checked behavior

One optimized native build and one bounded invocation pass all237 methods. Independent direct multiplication confirms the full479-input polynomial identity.729 word-boundary cases,27 repeated-offset ternary cases, zero/constant/empty input behavior, oracle failure order and modulus boundaries pass. All8 complete recovery receipts match the prior factored-source implementation.33 prebuild source/data hashes verify. The source is imported unchanged; only two recovery module routes differ.

Scope and next step

The evaluator changes; the supplied polynomial and its construction cost do not. No comparative panel or cloud job ran. Next count the whole decoder and setup, including word logic, signed accumulation and reductions, in a separately frozen matched comparison. Original4563-case cohort and previous panel identities remain intact.15 compiler warnings are retained.

Evidence and context

See DERIVATION.md, control.json, source-lock.json, receipt.json and raw logs. Established bit-serial arithmetic context: Umuroglu et al. (2019), sections2.1–2.2, read2026-11-03. Its FPGA performance is not a CPU result here; no novelty claim for bit slicing or popcount dot products is made.